

JS .this with Arrow function (=>)

The screenshot shows a code editor on the left and a browser console on the right. The code in the editor defines a 'student' object with a 'name' property 'Anil', an 'age' property 23, and a 'pro' property 'this'. It also has two methods: 'getName' (a regular function) and 'getAge' (an arrow function). The 'getName' method logs 'this' (the student object), 'this.name' ('Anil'), and 'hii !'. The 'getAge' method logs 'this' (the window object), 'this.age' (23), and 'hey'. The console on the right shows the output of these logs, with 'this' being the student object for 'getName' and the window object for 'getAge'.

```
7.) Day 20 js -- > this with arrow > JS this.js > ...
1 // this with arrow function
2 let student = {
3   //parrent ka this will be -- window object
4   name: "Anil",
5   age: 23,
6   pro: this,
7   getName: function () {
8     console.log(this); //iska this will be Calling object
9     console.log(this.name); // hence calling object is "student"
10    console.log("hii !"); // So, Student object is this.
11  },
12
13  getAge: () => {
14    console.log(this); //iska this parrent ka this hoga
15    console.log(this.age); // hence the parrent this is window object
16    console.log("hey"); // to iska this bhi - window object hi hoga
17  },
18 };
19
20 console.log(student.getName());
21 console.log(student.getAge());
22
23
24
25
```

Console output:

- this.js:8 {name: 'Anil', age: 23, pro: Window, getName: f, getAge: f}
- this.js:9 Anil
- this.js:10 hii !
- this.js:20 undefined
- this.js:14 Window {window: Window, self: Window, document: document, name: '', location: Location, ...}
- this.js:15 undefined
- this.js:16 hey
- this.js:21 undefined

```
let student = {
```

```
  //parrent ka this will be -- window object
```

```
  name: "Anil",
```

```
  age: 23,
```

```
  pro: this,
```

```
  getName: function () {
```

```
    console.log(this); //iska this will be Calling object
```

```
    console.log(this.name); // hence calling object is "student"
```

```
    console.log("hii !"); // So, Student object is this.
```

```
  },
```

```
  |
  getAge: () => {
```

```
    console.log(this); //iska this parrent ka this hoga
```

```
    console.log(this.age); // hence the parrent this is window object
```

```
    console.log("hey"); // to iska this bhi - window object hi hoga
```

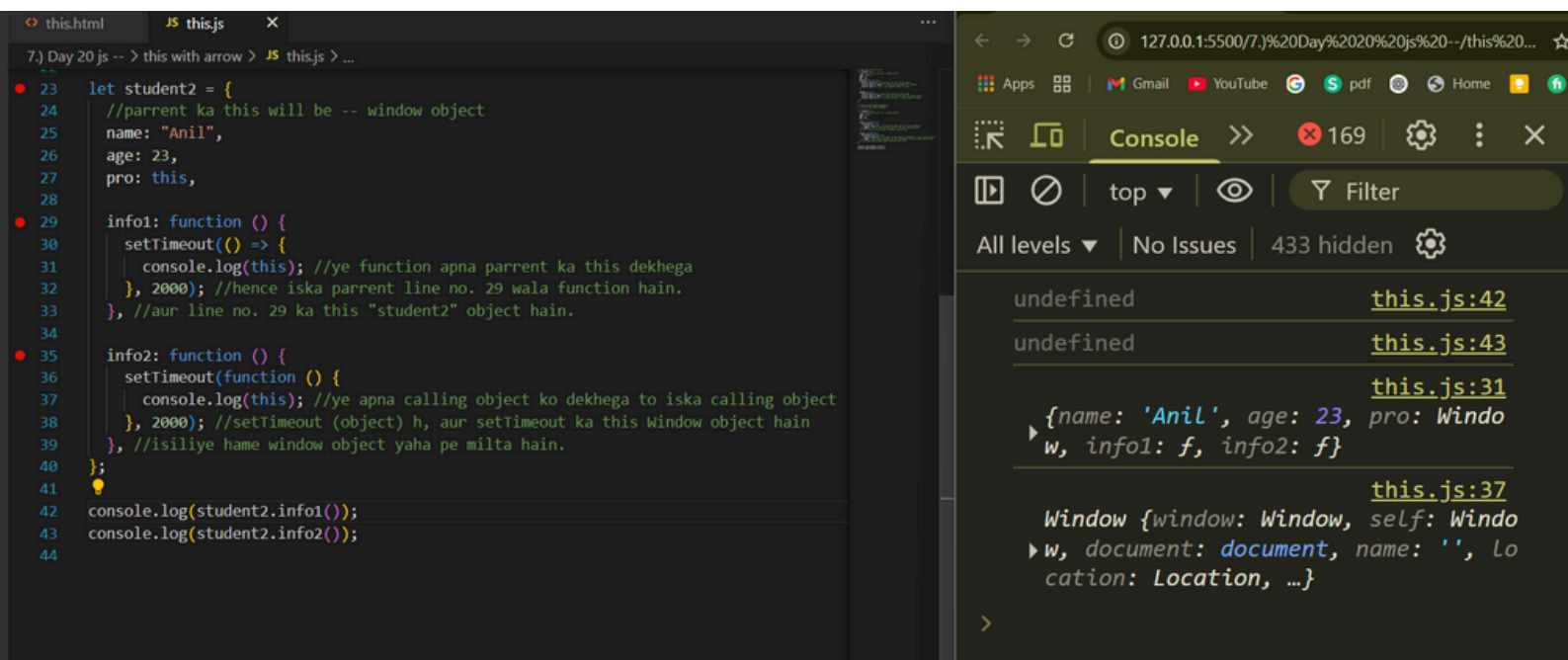
```
},
```

```
};
```

```
console.log(student.getName());
```

```
console.log(student.getAge());
```

Case ii



```
let student2 = {
```

```
  //parent ka this will be -- window object
```

```
  name: "Anil",
```

```
  age: 23,
```

```
  pro: this,
```

```
|
```

```
  info1: function () {
```

```
    setTimeout(() => {
```

```
      console.log(this); //ye function apna parrent ka this dekhega
```

```
    }, 2000); //hence iska parrent line no. 29 wala function hain.
```

```
  }, //aur line no. 29 ka this "student2" object hain.
```

```
|  
info2: function () {  
  
    setTimeout(function () {  
  
        console.log(this); //ye apna calling object ko dekhega to iska calling object  
  
    }, 2000); //setTimeout (object) h, aur setTimeout ka this Window object hain  
  
    }, //isiliye hame window object yaha pe milta hain.  
  
};
```

```
|  
console.log(student2.info1());  
  
console.log(student2.info2());  
  
|
```